

Seminar Attendance Registration System

Comprehensive Project Report & Technical Documentation

Student Name: Tanuj Sharad Thote

Department: Computer Science & Engineering (AI & ML)

Roll Number: MLU24F063

Division / Section: Section A

Course: DBMS

Academic Term: 2026

Hosted Application URL: <http://seminar-attendance-tanuj.rf.gd>

Cloud Provider: InfinityFree Hosting

2. Selected Problem Statement

Educational institutes, department seminars, and academic workshops frequently struggle with manually collecting, validating, and tracking student attendance records. Paper-based sign-in sheets are prone to errors, loss, illegible hand-writing, proxy entries, and inefficient querying when analytical logs are required by faculty members.

To resolve these limitations, this project implements a dynamic, web-based **Seminar Attendance Registration System**. The platform provides a secure digital portal for students to register attendance for specific academic seminars, stores structured attendance logs in a relational database engine, and equips administrators with automated dynamic search and filtering interfaces.

3. Project Objectives & Key Features

3.1 Primary Objectives

- Develop a full-stack, database-driven web application using standard web technologies (HTML, CSS, PHP, MySQL).
- Deploy the functional application to a cloud web server (InfinityFree) with public HTTP accessibility.
- Implement robust security controls using parameterized prepared statements to prevent SQL Injection (SQLi) and XSS vulnerabilities.
- Provide seamless data visualization and search capabilities for stored attendance records.

3.2 Core Application Features

- **Interactive Attendance Form:** Front-end web form equipped with validation rules for participant details.
- **Relational Data Persistence:** Real-time HTTP POST processing and secure database entry generation.
- **Attendance Log Viewer:** Tabular interface displaying existing logs with automated creation timestamps.
- **Multi-Criteria Search Engine:** Real-time dynamic search feature allowing lookups by student name, roll number, or seminar title.

- **Defensive SQL Security:** Complete reliance on prepared statements and parameter binding across all database queries.

4. Technology Stack & Deployment Environment

Layer / Subsystem	Technology	Implementation Role & Specifications
Front-End Interface	HTML5 & CSS3	Provides structured input forms (`index.php`), clean responsive navigation links, table layouts (`view.php`), and custom visual styling (`style.css`).
Back-End Scripting	PHP 8.x	Executes server-side form handling, session validation, parameter sanitization, and database query processing (`save.php`, `search.php`).
Database Tier	MySQL 8.0 / MariaDB	Stores participant attendance records in table `seminar_attendance` hosted on MySQL host `sql310.infinityfree.com`.
Cloud Web Server	InfinityFree (Apache)	Public cloud hosting environment hosting PHP files inside the virtual root `/htdocs` directory.
Database Management	phpMyAdmin	Web interface utilized for schema migration, table creation, and execution of `database.sql` script.

5. Database Table Design & Constraints

The application utilizes a MySQL database named `if0_42854369_attendance`. The transactional records are stored inside the `seminar_attendance` table designed with strict column constraints.

Column Name	Data Type	Key / Attributes	Constraints & Description
<code>id</code>	INT	PRIMARY KEY	AUTO_INCREMENT, Unique record identifier.
<code>participant_name</code>	VARCHAR(100)	NOT NULL	Full name of the student/attendee. Cannot be null.
<code>roll_number</code>	VARCHAR(30)	NOT NULL	Academic roll number (e.g., `MLU24F063`). Indexed for fast lookups.
<code>seminar_title</code>	VARCHAR(150)	NOT NULL	Official title or code of the attended seminar session.
<code>attendance_state</code>	ENUM	NOT NULL	Restricted values: `Present`, `Absent`, or `Late`.
<code>created_at</code>	TIMESTAMP	DEFAULT	`CURRENT_TIMESTAMP`. Automatically logs entry submission time.

6. Entity-Relationship (ER) Schema

Below is the visual Entity-Relationship layout representing the relational model for the attendance domain:

ENTITY: seminar_attendance

```
+-----+
| id (PK) : INT (11) AUTO_INCREMENT |
| participant_name : VARCHAR(100) NOT NULL |
| roll_number : VARCHAR(30) NOT NULL |
| seminar_title : VARCHAR(150) NOT NULL |
| attendance_state : ENUM('Present','Absent','Late') |
| created_at : TIMESTAMP DEFAULT CURRENT_TIMESTAMP |
+-----+
```

```
-- SQL Definition Script (database.sql)
CREATE TABLE IF NOT EXISTS `seminar_attendance` (
  `id` INT AUTO_INCREMENT PRIMARY KEY,
  `participant_name` VARCHAR(100) NOT NULL,
  `roll_number` VARCHAR(30) NOT NULL,
  `seminar_title` VARCHAR(150) NOT NULL,
  `attendance_state` ENUM('Present', 'Absent', 'Late') NOT NULL,
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

7. End-to-End Application Flow Explanation

The system follows a classical 3-Tier Web Application Architecture (Client / Application / Database). The lifecycle of a registration request operates as follows:

Step 1: Front-End User Form Interaction (`index.php`)

The user accesses the web portal via browser and fills out the HTML5 form specifying their Name, Roll Number, Seminar Title, and Attendance Status.

Step 2: HTTP POST Request Dispatch

Upon clicking "Submit Attendance", an HTTP POST request carrying payload data is sent to the server-side target script `save.php`.

Step 3: Database Connection & Verification (`config.php`)

`save.php` includes `config.php`, which initializes a MySQL connection via `mysqli\_connect()`. Credentials (`sql310.infinityfree.com`, `if0\_42854369`) are authenticated.

Step 4: Parameter Binding & SQL Execution

PHP prepares the SQL template statement (`INSERT INTO seminar\_attendance...`). Parameters are safely bound, preventing SQL syntax manipulation before sending the query execution command to MySQL.

Step 5: Database Persistence & Feedback Output (`save.php` / `view.php`)

MySQL writes the record into persistent storage. PHP returns an HTML success message back to the user browser, providing hyperlinks to view updated records on `view.php` or query logs on `search.php`.

8. List of SQL Operations Implemented

The application implements fundamental CRUD SQL operations to interact with the database:

8.1 Data Insertion (`INSERT`)

Executed inside `save.php` to insert newly submitted form records:

```
INSERT INTO seminar_attendance (participant_name, roll_number, seminar_title,
attendance_state)
VALUES (?, ?, ?, ?);
```

8.2 Data Retrieval (`SELECT`)

Executed inside `view.php` to render all registered participants in chronological order:

```
SELECT id, participant_name, roll_number, seminar_title, attendance_state, created_at
FROM seminar_attendance
ORDER BY id DESC;
```

8.3 Filtered Search Queries (`SELECT` with `WHERE` & `LIKE`)

Executed inside `search.php` to filter attendance records dynamically based on user input:

```
SELECT * FROM seminar_attendance
WHERE participant_name LIKE ? OR roll_number LIKE ? OR seminar_title LIKE ?;
```

8.4 Update and Delete Operations

Administrative maintenance queries structured for record lifecycle operations:

```
-- UPDATE Statement (Modify Attendance State)
UPDATE seminar_attendance
SET attendance_state = ?
WHERE id = ?;

-- DELETE Statement (Remove Duplicate Record)
DELETE FROM seminar_attendance
WHERE id = ?;
```

9. Prepared Statements & Defensive Security Explanation

In standard dynamic SQL concatenation, user input strings are directly appended into query strings (e.g., `"SELECT * FROM users WHERE name = '" . $userInput . "'"`). If an attacker enters malicious input such as `' OR '1'='1`, the SQL interpreter parses the input as executable code, leading to severe **SQL Injection (SQLi)** vulnerabilities.

Why Prepared Statements Are Crucial for Application Security

A **Prepared Statement** separates the SQL query logic from the input data through a two-stage process:

- Compilation Phase:** The database engine compiles the SQL query template containing parameter placeholders (?) into an Execution Tree.
- Execution Phase:** User inputs are bound strictly as literal parameter values. The database engine never interprets bound data as executable SQL commands, completely neutralizing SQL Injection threats.

10. Hosted Application Details

Attribute	Hosting Configuration Details
Live Web Domain URL	http://seminar-attendance-tanuj.rf.gd
Web Server Engine	Apache HTTP Server on Linux Operating System
MySQL Database Host	sql310.infinityfree.com
Database Name	if0_42854369_attendance

11. Functional Test Cases & Validation Summary

Test Case ID	Module Tested	Input Data / Action	Expected Output	Status
TC-01	Form Submission	Submit valid participant details via `index.php`.	Record saved to MySQL; confirmation displayed.	PASSED
TC-02	Data Visualization	Navigate to `view.php`.	All DB records rendered inside responsive HTML table.	PASSED
TC-03	Search Filter	Enter search keyword "Tanuj" in `search.php`.	Table filters to display matching student records only.	PASSED
TC-04	SQL Security Test	Submit participant name Robert O'Reilly.	Name saved and displayed safely without single-quote SQL syntax error.	PASSED

11.1 Screenshot Verification Index

The submission archive includes the following 7 verification screenshots captured from the live deployment environment:

- `1_homepage.png` – Live attendance registration form interface on ``index.php``.
- `2_form_filled.png` – Form filled with sample participant data prior to submission.
- `3_submit_confirmation.png` – Success confirmation message output after executing ``save.php``.
- `4_view_records.png` – Full attendance table view on ``view.php``.
- `5_search_feature.png` – Filtered search query results rendered on ``search.php``.
- `6_security_test.png` – Special character validation proof displaying ``Robert O'Reilly`` safely.
- `7_incognito_url.png` – Incognito browser window verifying live URL domain accessibility.

12. Declaration of Original Work

I, **Tanuj Sharad Thote**, student of **Computer Science & Engineering (Artificial Intelligence and Machine Learning)**, bearing Roll Number **MLU24F063**, hereby declare that the project titled **"Seminar Attendance Registration System"** and this accompanying project report represent my own original academic work.

All source code, database architecture, security tests, and cloud deployments documented herein were developed and executed by me for the Web Application Architecture Lab assignment. No part of this submission has been plagiarized or copied from unauthorized external sources.

Date: September 7, 2026
Place: Amravati, Maharashtra

Tanuj Sharad Thote
Roll No: MLU24F063